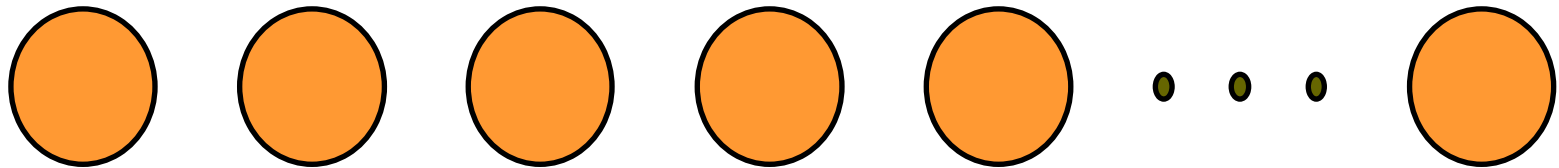


Estrutura de Dados

LISTA LINEAR SEQUENCIAL

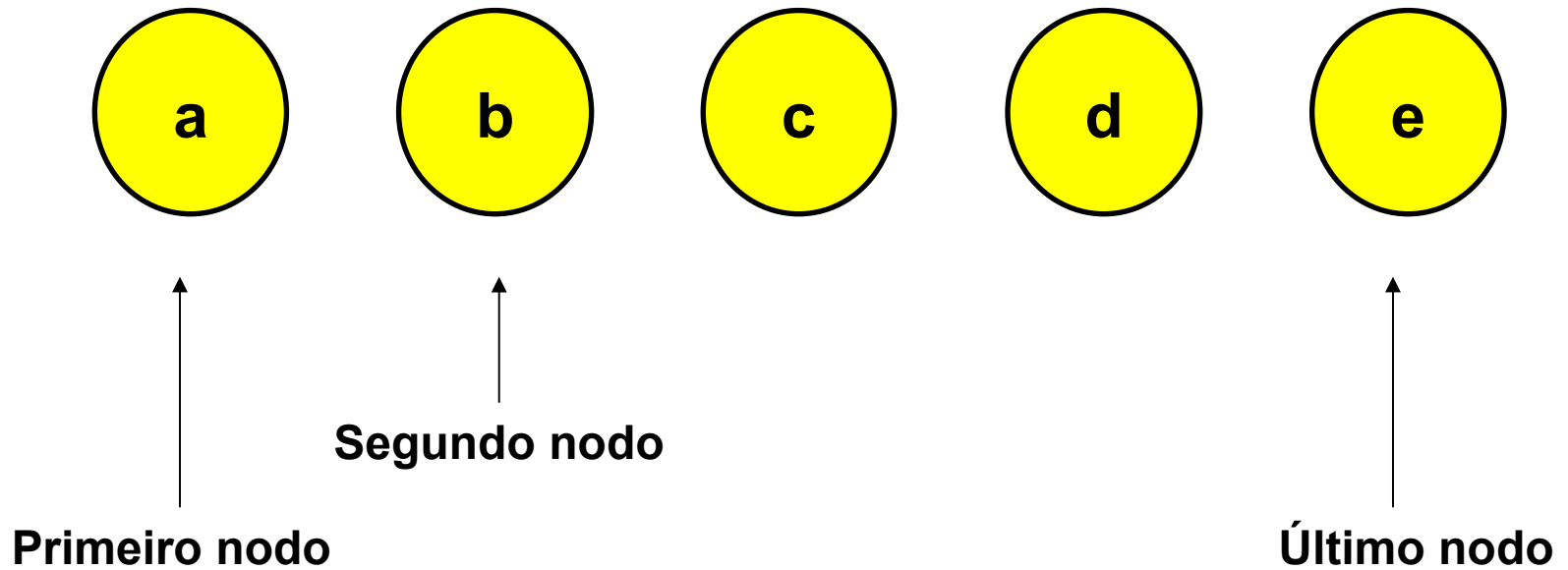
Lista linear

Uma Lista Linear (LL) é uma **sequencia** de nodos



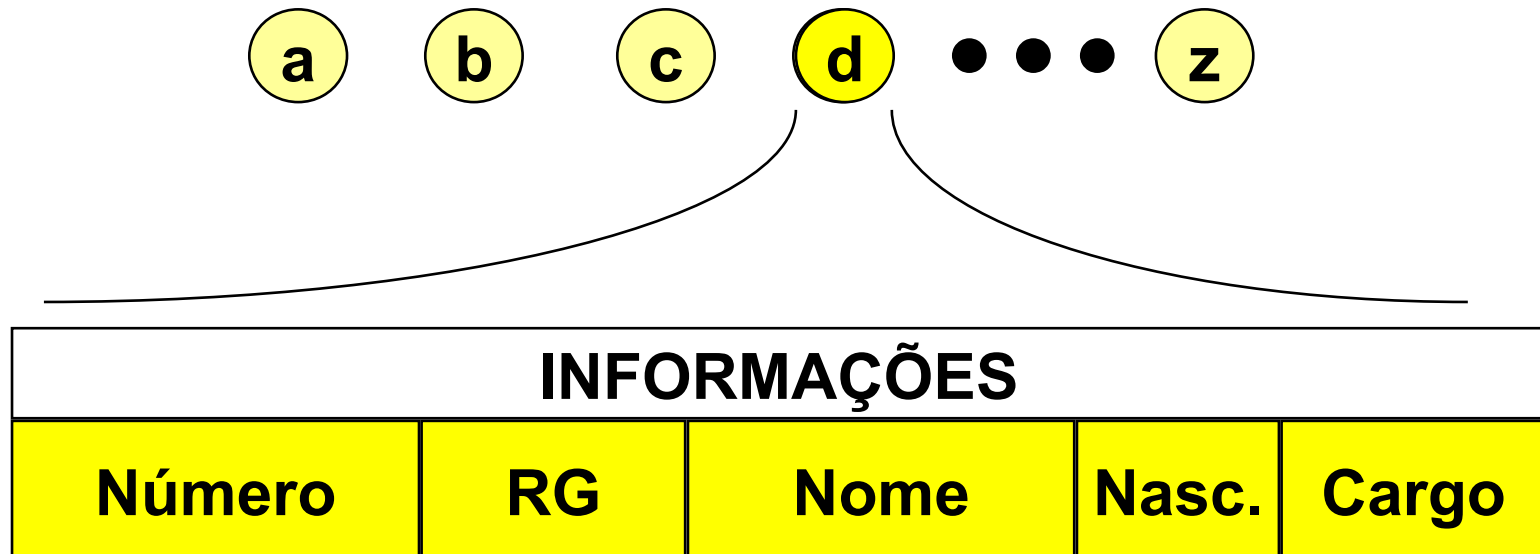
- **Nodos - elementos do mesmo tipo**
- **Relação de ordem linear (ou total)**

Lista linear



Estrutura dos nodos

- Estrutura interna é abstraída



Definição formal

Uma lista linear é uma coleção de $n \geq 0$ nodos $x_1, x_2, \dots, x_n$, todos do mesmo tipo, cujas propriedades estruturais relevantes envolvem apenas as posições relativas lineares entre nodos:

$n = 0$: lista vazia, apresenta zero nodos

$n > 0$: x_1 é o primeiro nodo

x_n é o último nodo

$1 < k < n$: x_k é precedido por x_{k-1} e sucedido por x_{k+1}

- Lista linear: sequência de 0 ou mais nodos do mesmo tipo

Exemplos de aplicações com listas

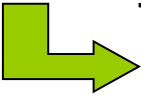
- **Notas de alunos**
- **Cadastro de funcionários de uma empresa**
- **Itens em estoque em uma empresa**
- **Dias da semana**
- **Vagões de um trem**
- **Letras de uma palavra**
- **Pessoas esperando ônibus**
- **Cartas de baralho**
- **Precipitações pluviométricas em um mês / dia**

Operações sobre listas lineares

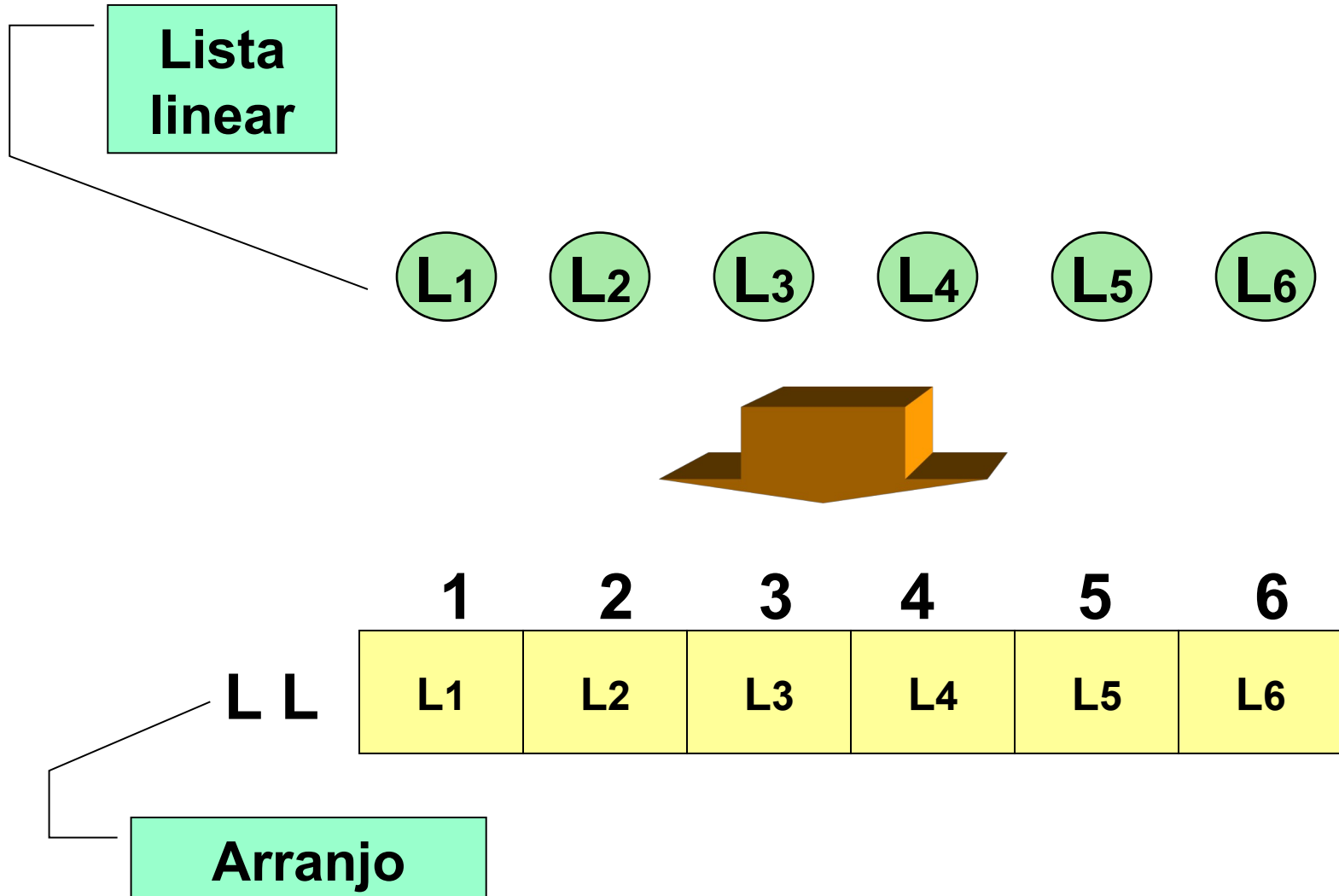
Operações básicas:

- **Criação de uma lista**
- **Inserção de um nodo**
- **Exclusão de um nodo**
- **Acesso a um nodo**
- **Destruição de uma lista**

Lista Linear Sequencial

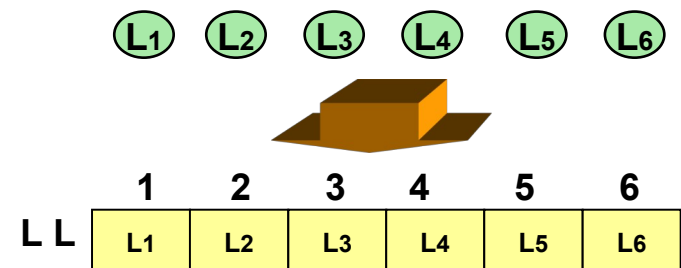
- Utiliza a seqüencialidade da memória para representar a ordem entre os nodos
- Endereços fisicamente adjacentes
 nodos logicamente adjacentes

Lista Linear Sequencial



Características

- todos os elementos do arranjo tem tipo igual
- cada elemento do arranjo é um nodo
- tipo do nodo = tipo do elemento do arranjo
- posição no arranjo representa posição na LL
- acesso direto aos nodos
- natureza dinâmica – inserção / remoção de nodos
- comprimento varia durante aplicação



Tipos de dados

Tipos de dados utilizados nos algoritmos para LL implementada com sequencial:

```
TipoNode = registro
    Nome: string
    Código: inteiro
    Valor: real
fim registro
```

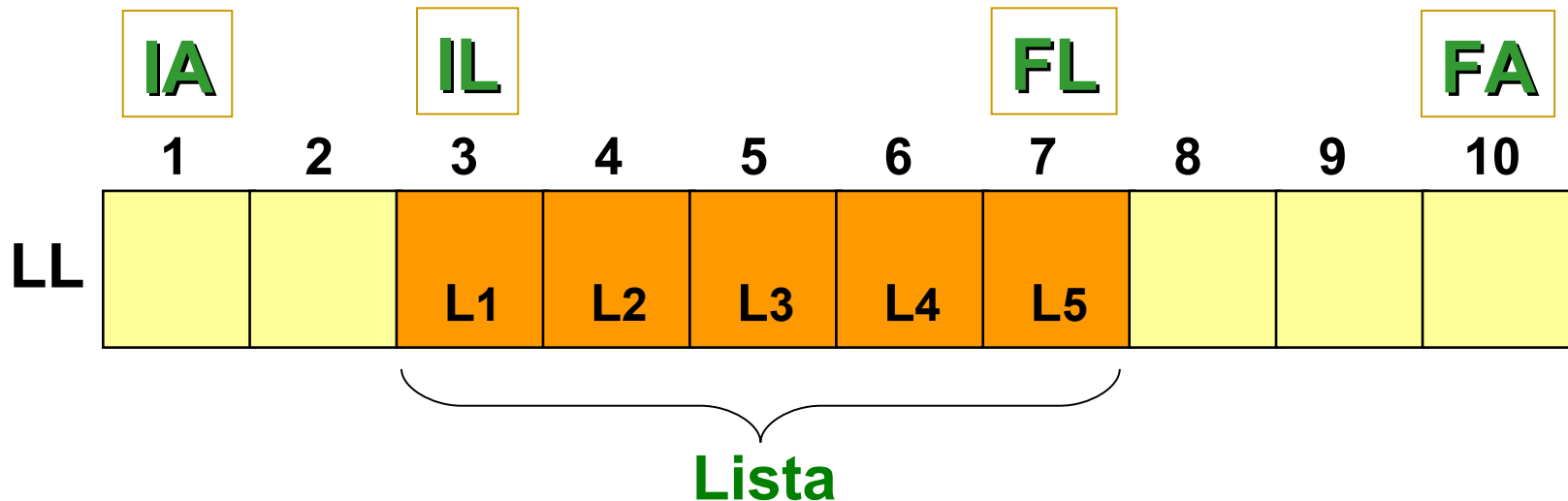
```
TipoLista = arranjo [1 .. N] de TipoNode
```

Início e final da LL

Início e final da LL geralmente são diferentes do início e final do arranjo

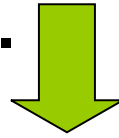


Variáveis para identificar **Início e Final da Lista**

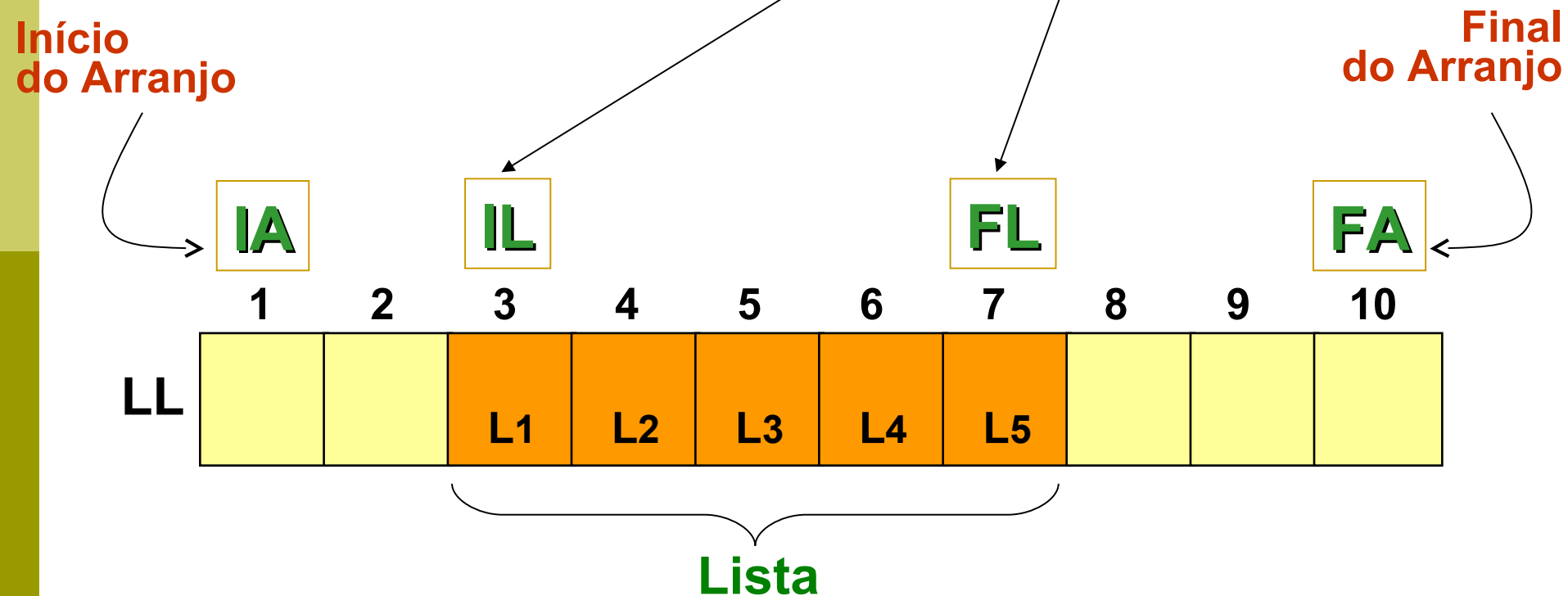


Início e final da LL

Início e final da LL geralmente são diferentes do início e final do arranjo (vetor).

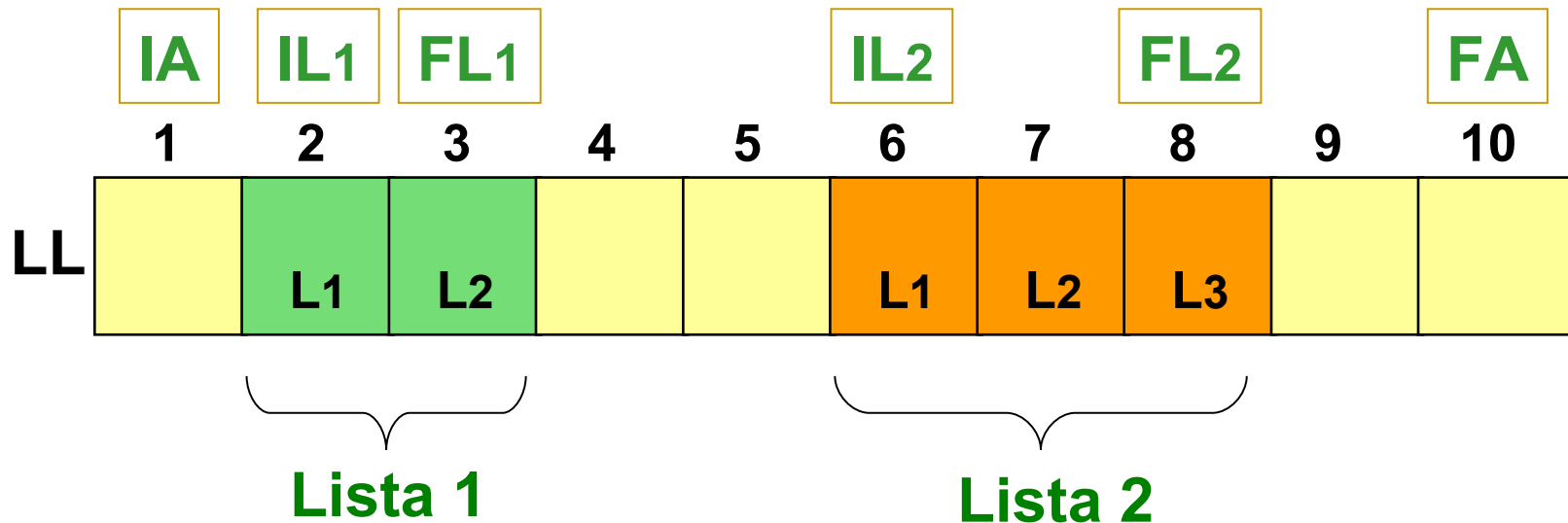


Variáveis para identificar **Início e Final da Lista**



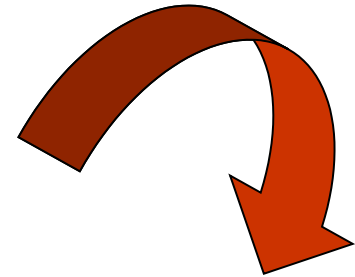
Mais de uma LL em um vetor

Um mesmo vetor pode ser utilizado para mais de uma lista linear



Operações sobre listas lineares sequenciais

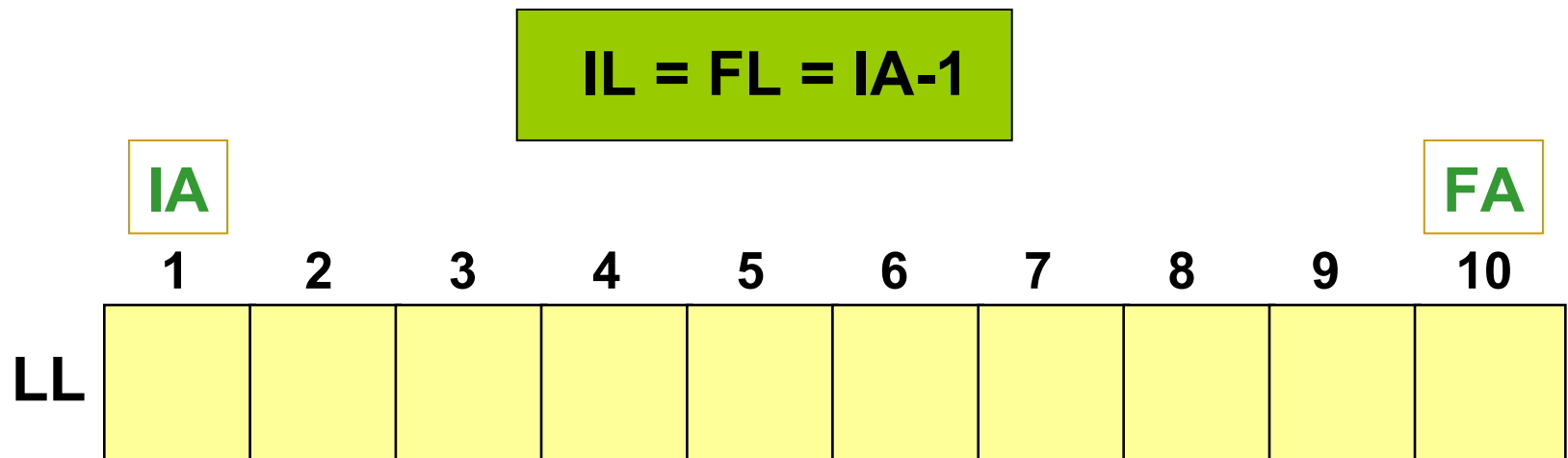
- criação da lista
- inserção de um novo nodo
- remoção de um nodo
- acesso a um nodo



Algoritmos

Criação de uma LL vazia

- Declarar o arranjo
- Inicializar variáveis que guardam início e final da lista



Lista vazia

Algoritmo:

Inicializar LL sobre Arranjo vazio

Algoritmo **InicializarLLArr**

Entradas: IA (inteiro)

Saídas: IL, FL (inteiro)

início

IL $\leftarrow$ FL $\leftarrow$ IA - 1

fim

Ex: Lista de informações de produtos de uma empresa

```
TipoNode = registro
```

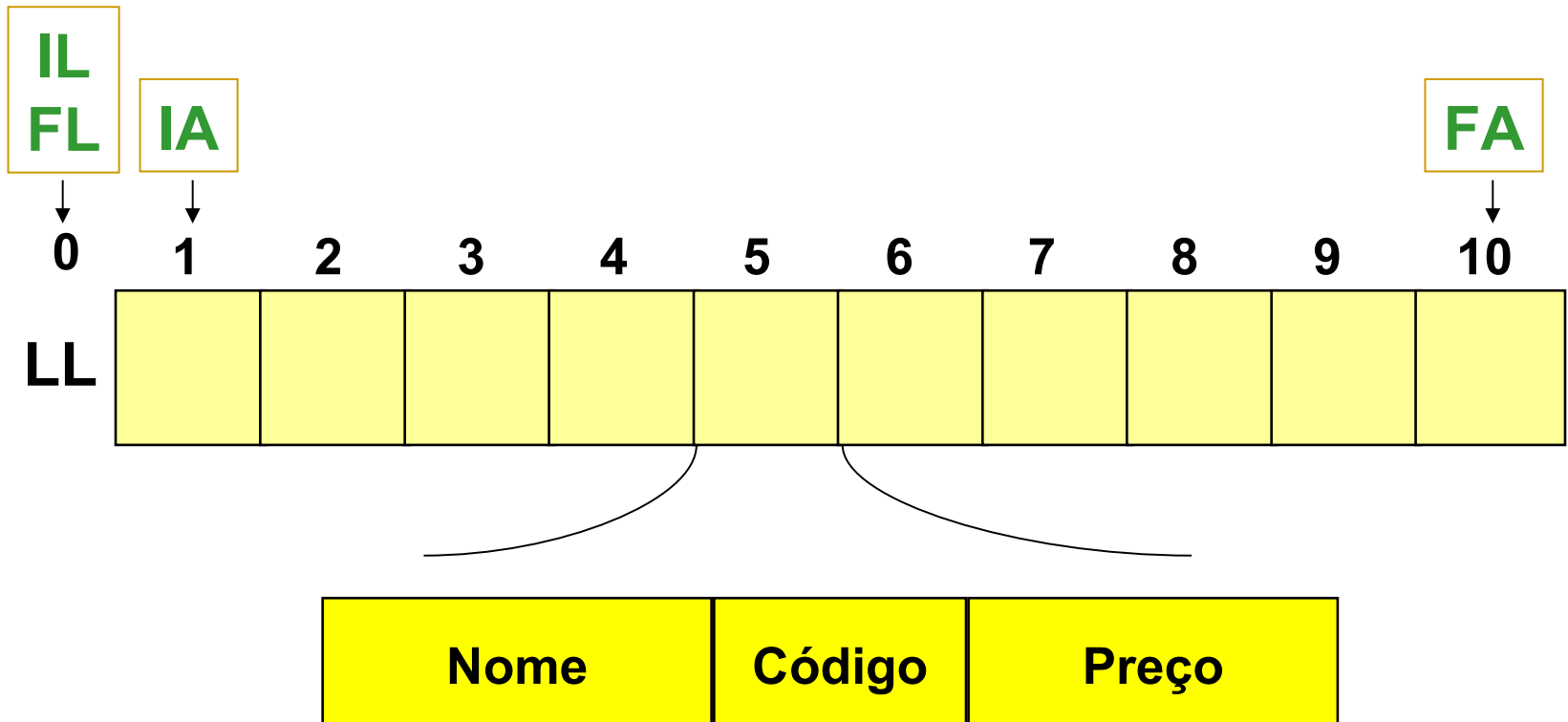
```
    Nome: string
```

```
    Código: inteiro
```

```
    Preço: real
```

```
    fim registro
```

```
TipoLista = arranjo [1 .. 10] de TipoNode
```



Inserção de um novo nodo

➔ LL já existente

	IA 0	IL 1	2	3	4	5	FL 6	FA 7
LL		L1	L2	L3	L4	L5	L6	

Informar:

- posição (ordem) do novo nodo na LL
- valor do campo de informação do novo nodo

Quando não é possível realizar a inserção:

- posição solicitada fora dos limites da LL existente
- espaço disponível para a LL no arranjo todo preenchido



No início

A



No final

A

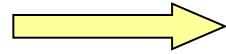


No meio

A

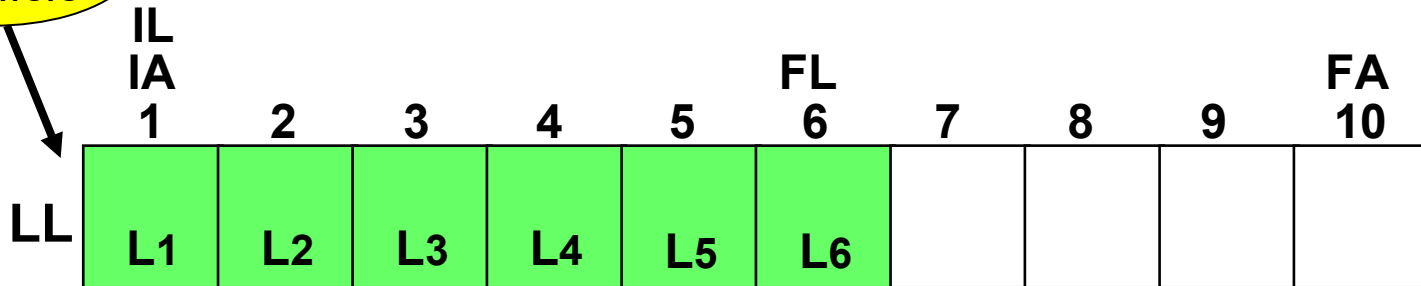


Inserção como primeiro nodo da LL

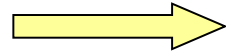


LL inicia no início do arranjo

Inserir
no início

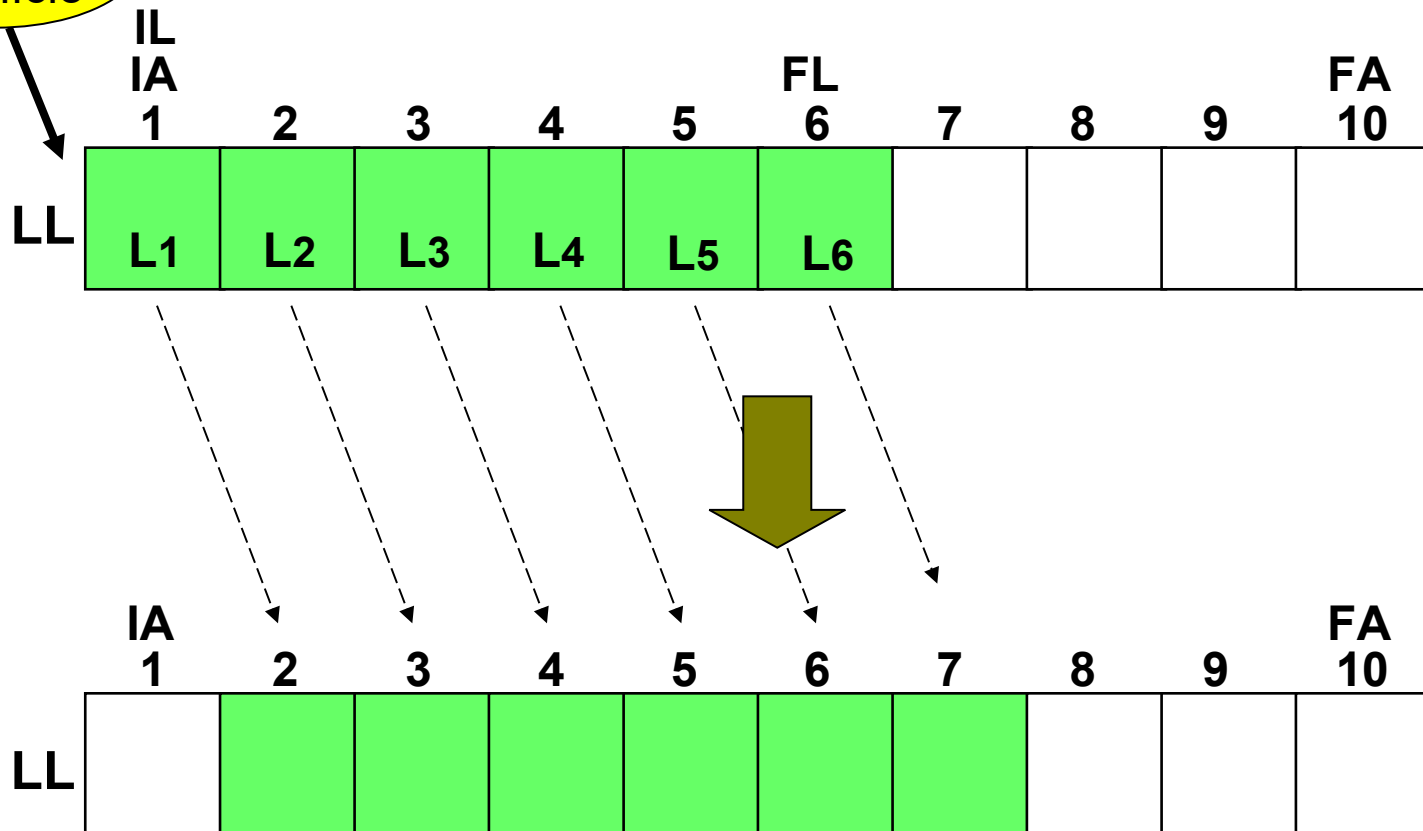


Inserção como primeiro nodo da LL

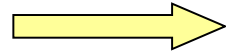


LL inicia no início do arranjo

Inserir
no início

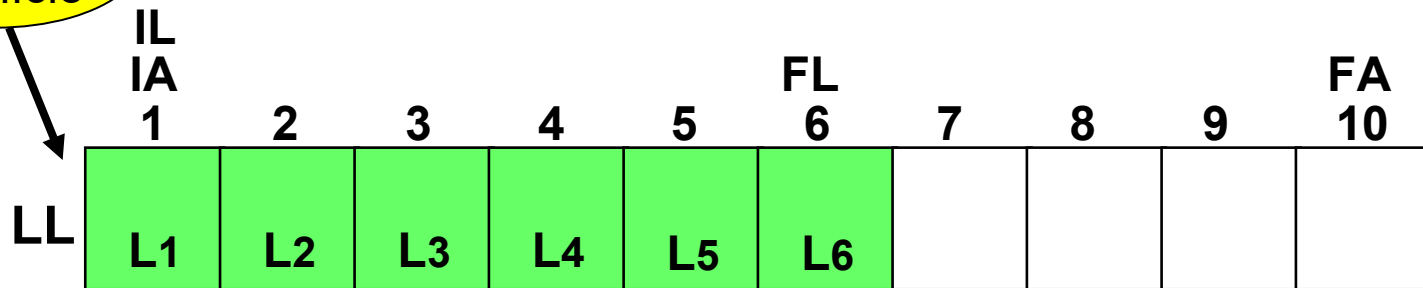


Inserção como primeiro nodo da LL

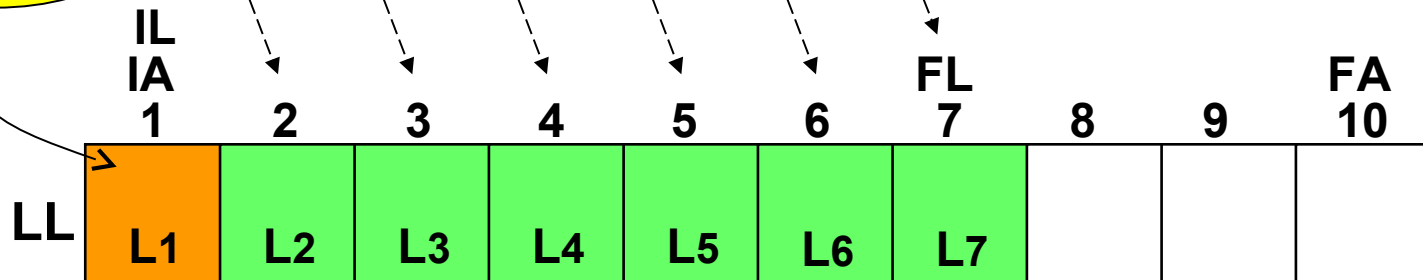


LL inicia no início do arranjo

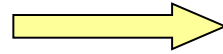
Inserir
no início



Novo nodo

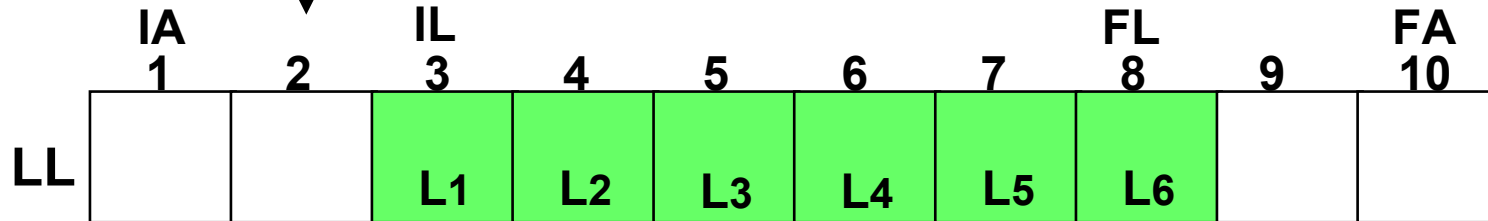


Inserção como primeiro nodo da LL

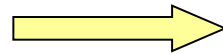


LL não inicia no início do arranjo

Inserir
no início

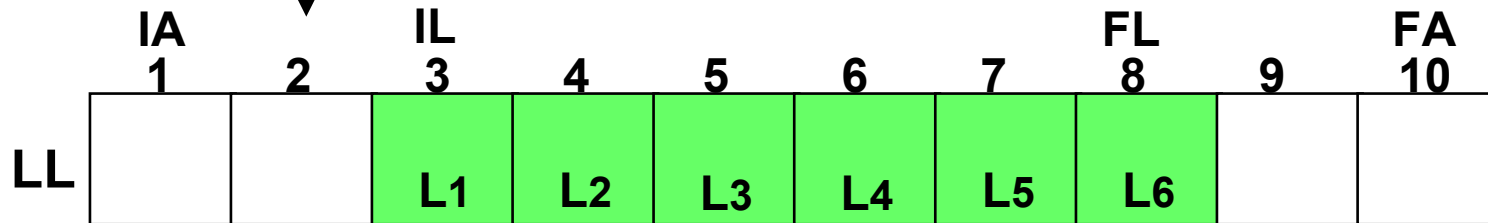


Inserção como primeiro nodo da LL

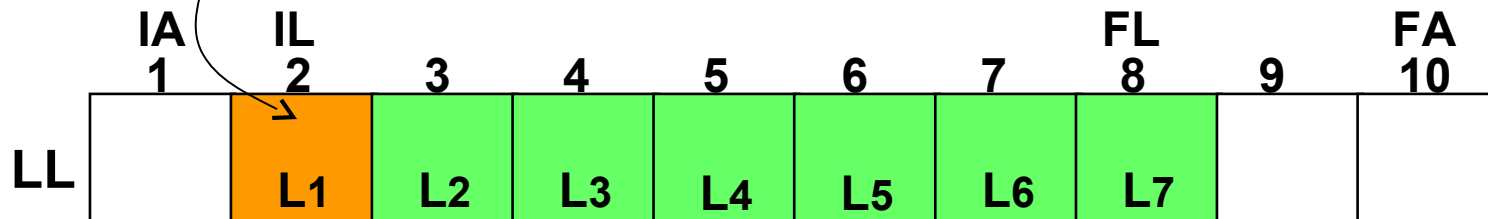
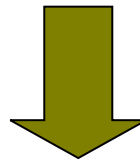


LL não inicia no início do arranjo

Inserir
no início



Novo nodo



Algoritmo:

Inserir no Início de LL implementada sobre Arranjo

Algoritmo InserirIniLLArr

Entradas: LL (TipoLista)
 IA, FA, IL, FL (inteiro)
 InfoNodo (TipoNodo)

Saídas: LL (TipoLista)
 IL, FL (inteiro)
 Sucesso (lógico)

Variável auxiliar: Ind (inteiro)

início

se (IA = IL) e (FA = FL) então {arranjo vazio}

Sucesso ← falso

senão

início

se IL = 0 então {lista vazia}

IL ← FL ← IA

senão

se IL > IA então {a lista não inicia na primeira posição do arranjo}

IL ← IL-1

senão

início {Deslocar nodos para cima}

para Ind de FL incr -1 até IL faça

LL[Ind+1] ← LL[Ind]

FL ← FL+1

fim

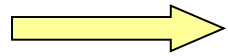
LL[IL] ← InfoNodo

Sucesso ← verdadeiro

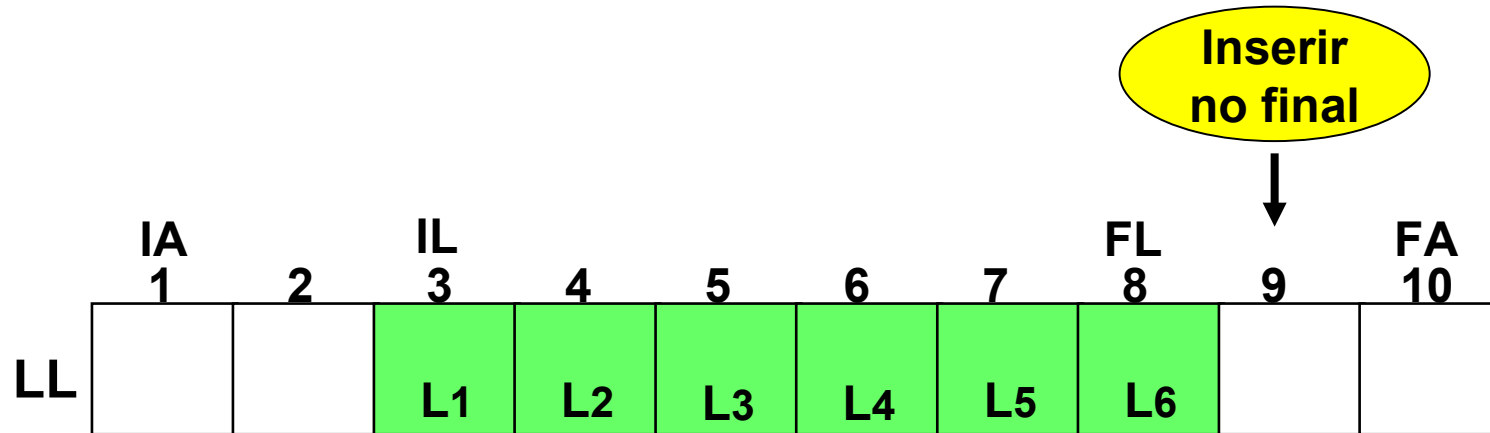
fim

fim

Inserção como último nodo da LL

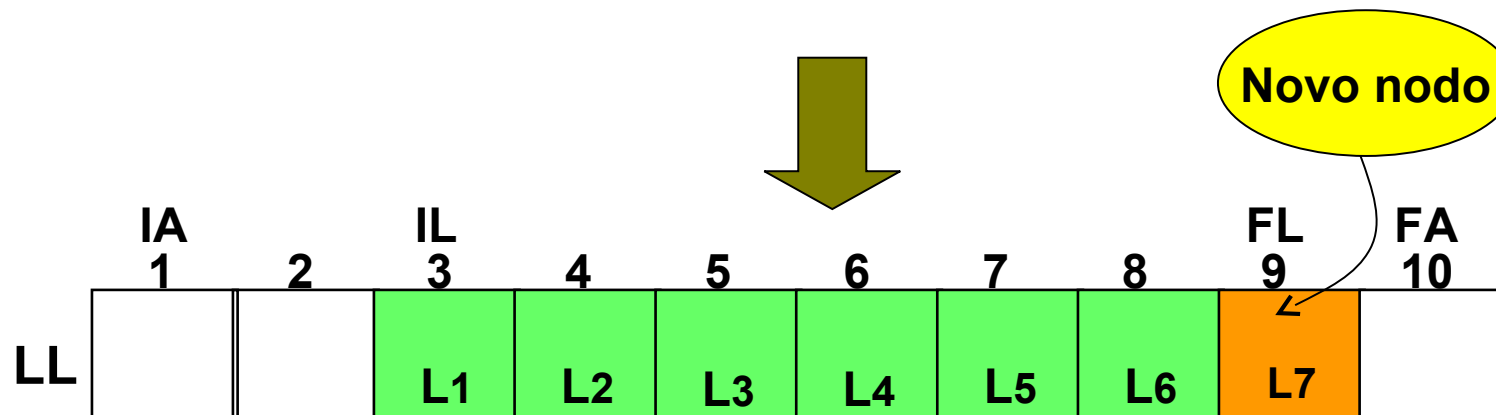
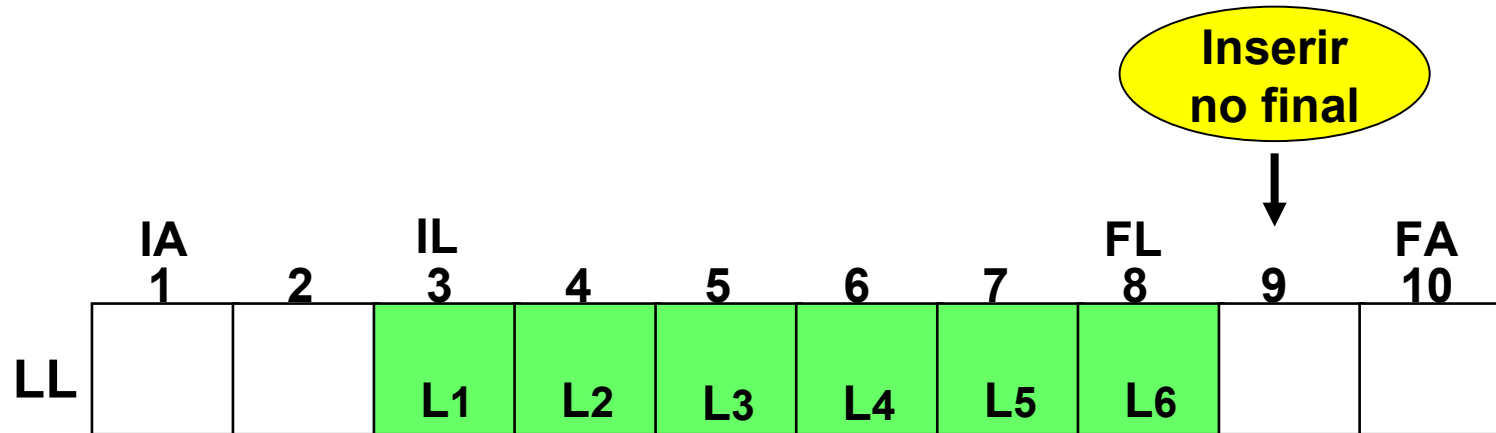


LL não termina no final do arranjo

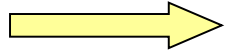


Inserção como último nodo da LL

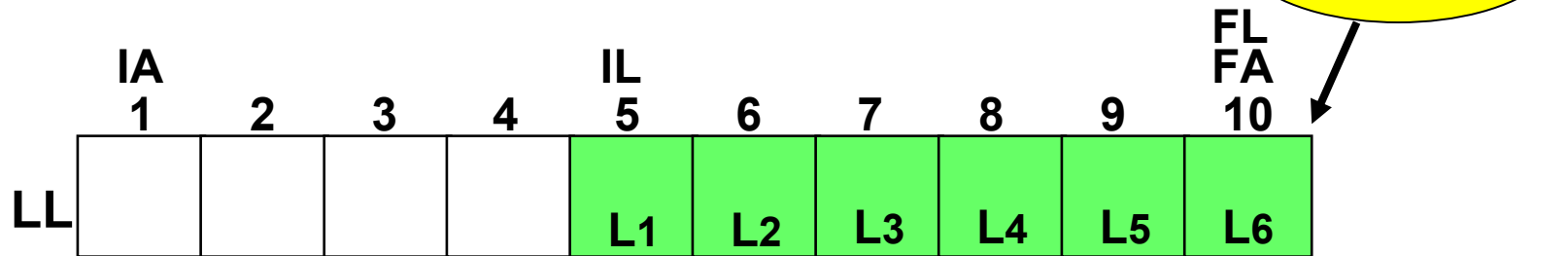
→ LL não termina no final do arranjo



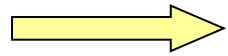
Inserção como último nodo da LL



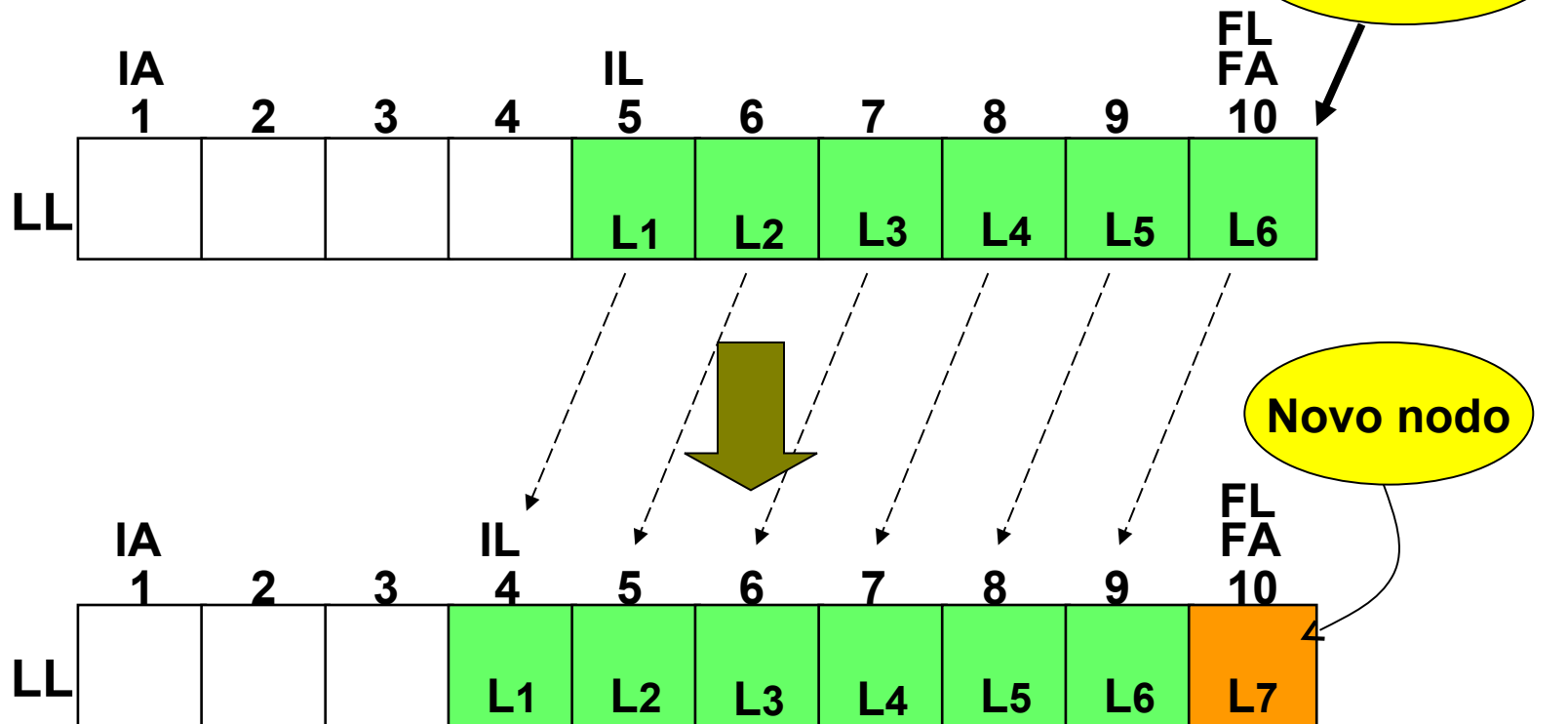
LL termina no final do arranjo



Inserção como último nodo da LL



LL termina no final do arranjo



Algoritmo:

Inserir no Fim de LL implementada sobre Arranjo

Algoritmo **InserirFimLLArr**

Entradas: LL (TipoLista)
IA, FA, IL, FL (inteiro)
InfoNodo (TipoNodo)

Saídas: LL (TipoLista)
IL, FL (inteiro)
Sucesso (lógico)

Variável auxiliar: Ind (inteiro)

início

se (IA = IL) e (FA = FL)

então Sucesso ← falso

senão início

se IL = 0 {LISTA VAZIA}

então IL ← FL ← IA

senão se FL < FA {TEM ESPAÇO ATRÁS}

então FL ← FL+1

senão início {DESLOCAR NODOS PARA BAIXO}

para Ind de IL incr 1 até FL faça

LL[Ind-1] ← LL[Ind]

IL ← IL-1

fim

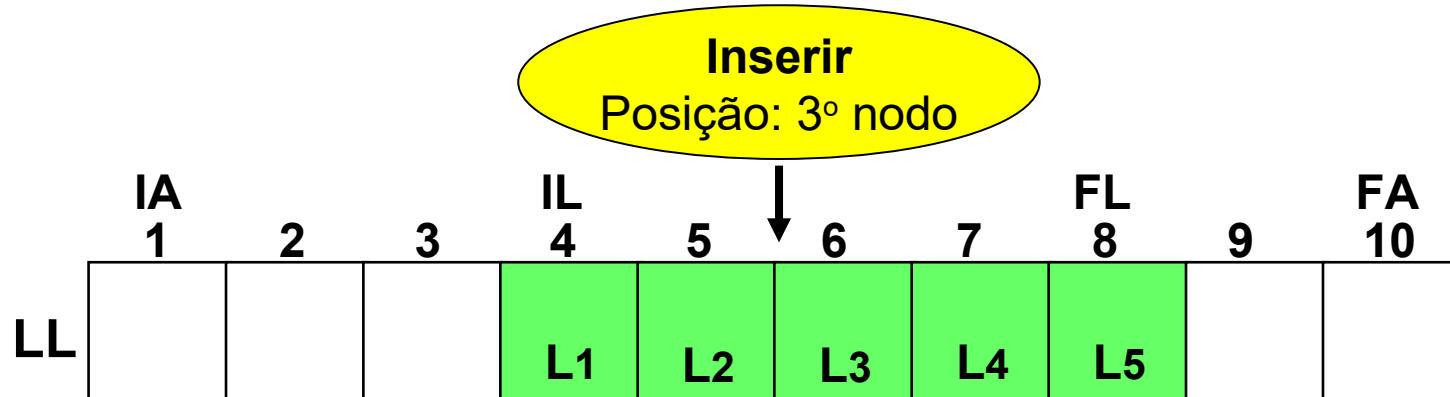
LL[FL] ← InfoNodo

Sucesso ← verdadeiro

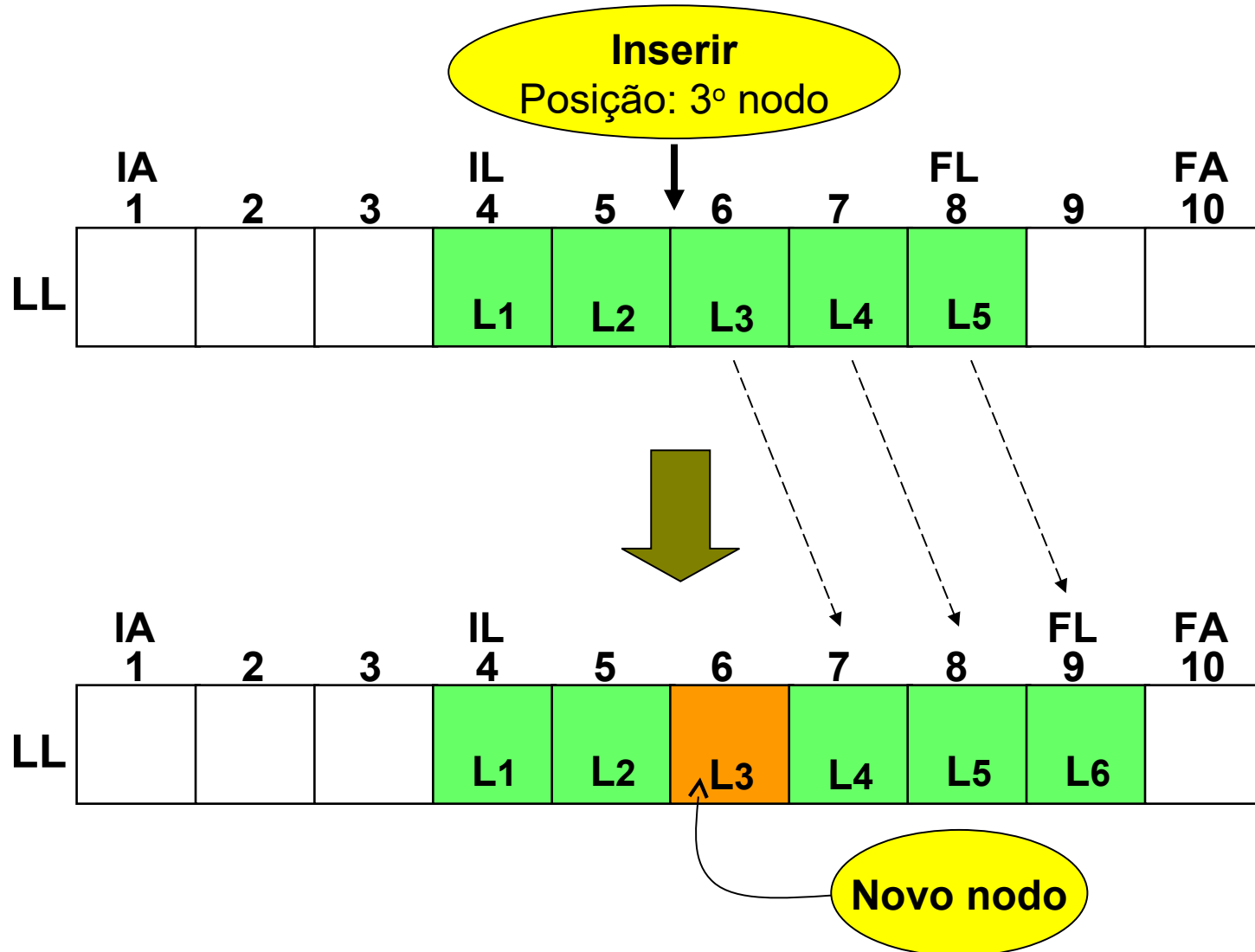
fim

fim

Inserção no meio de LL



Inserção no meio de LL



Algoritmo:

Inserir em LL implementada sobre Arranjo na Posição K

Posição K

- $K = 1 \rightarrow$ no início da LL
- $K = FL - IL + 2 \rightarrow$ no final da lista

Caso haja deslocamento

- para cima, caso haja espaço
- senão, para baixo

Sucesso retorna falso quando

- não existe espaço no arranjo para a inserção
- posição inválida para inserção na LL

Algoritmo:

Inserir em LL implementada sobre Arranjo na Posição K

Algoritmo InserirLLArrPosK

Entradas: LL (TipoLista)
 IA, FA, IL, FL (inteiro)
 K (inteiro)
 InfoNodo (TipoNodo)

Saídas: LL (TipoLista)
 IL, FL (inteiro)
 Sucesso (lógico)

Variável auxiliar: Ind (inteiro)

início

se (IA = IL e FA = FL) ou ($K > FL - IL + 2$) ou ($K \leq 0$) ou (IL = 0 e $K \neq 1$)

então Sucesso ← falso

senão início

se IL = 0

então IL ← FL ← IA

senão se FL ≠ FA

então início {DESLOCAR NODOS PARA CIMA}

para Ind de FL incr -1 até IL+K-1 faça

LL[Ind+1] ← LL[Ind]

FL ← FL+1

fim

senão início {DESLOCAR NODOS PARA BAIXO}

para Ind de IL incr 1 até IL+K-2 faça

LL[Ind-1] ← LL[Ind]

IL ← IL - 1

fim

LL[IL+K-1] ← InfoNodo

Sucesso ← verdadeiro

fim

fim

A posição não poderá ser maior do que o número de nodos da lista antes da inserção

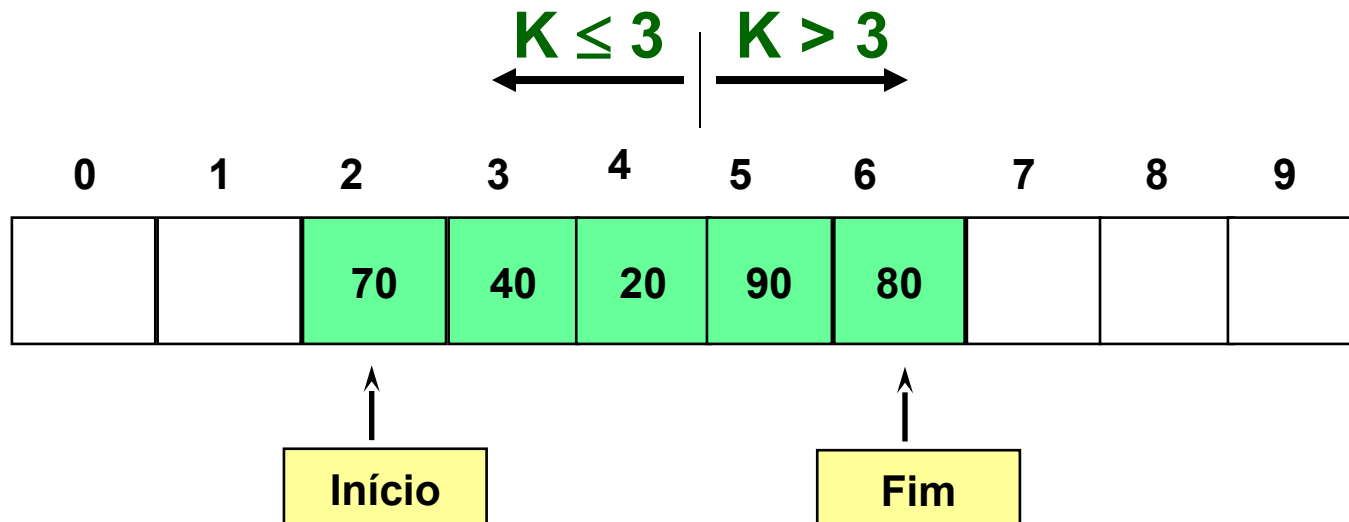
Algoritmo:

Inserir em LL implementada sobre Arranjo na Posição K

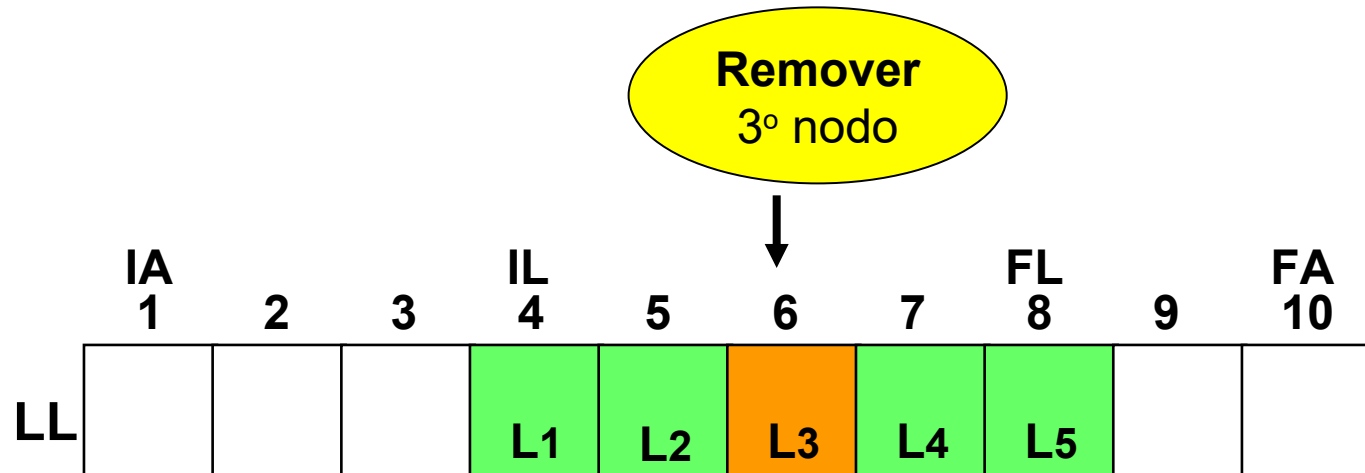
Otimizar:

deslocar nodos da **metade para o fim** se a inserção for na primeira metade da lista, e da **metade para o início** se for na segunda metade

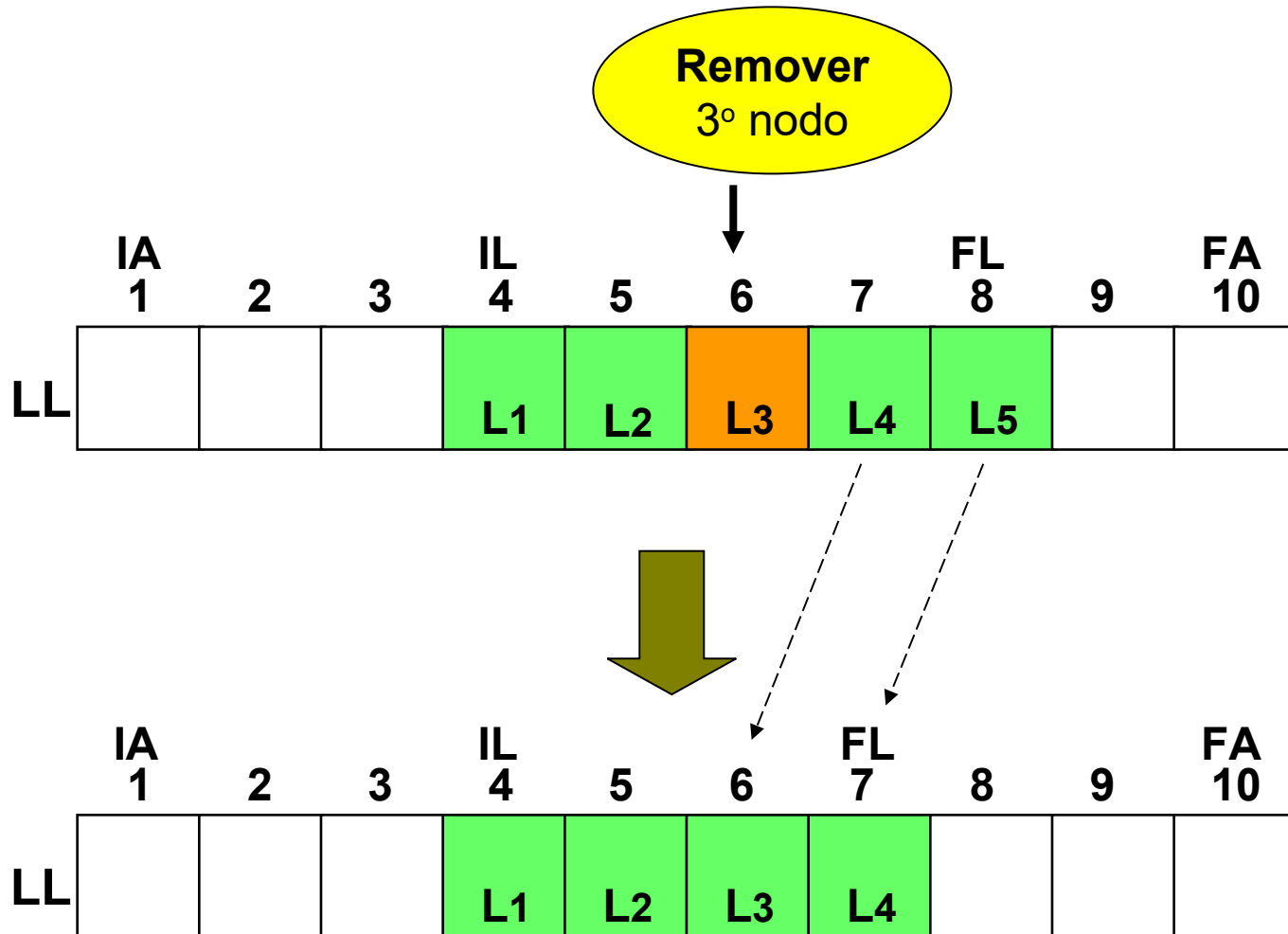
→ desde que haja espaço no lado considerado



Remoção de um nodo



Remoção de um nodo



Algoritmo:

Remover nodo de ordem K de LL implementada sobre Arranjo

Algoritmo **RemoverKLLArr**

Entradas: LL (TipoLista)
 IL, FL (inteiro)
 K (inteiro)

Saídas: LL (TipoLista)
 IL, FL (inteiro)
 Sucesso (lógico)

Variável auxiliar: Ind (inteiro)

início

se $(K \leq 0)$ ou $(K > FL - IL + 1)$

então Sucesso $\leftarrow$ falso

senão início

para Ind de $IL + K - 1$ incr 1 até $FL - 1$ faça

$LL[Ind] \leftarrow LL[Ind + 1]$

FL $\leftarrow$ FL - 1

se FL = IL - 1

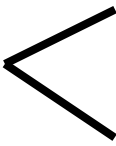
 então IL $\leftarrow$ FL $\leftarrow$ 0

Sucesso $\leftarrow$ verdadeiro

fim

fim

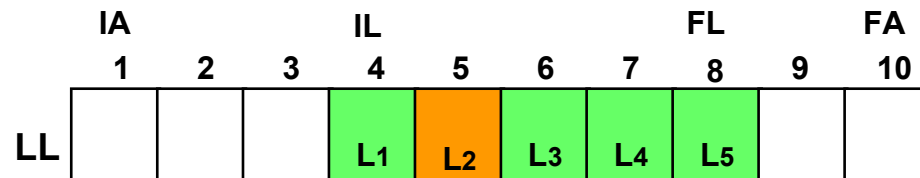
Acesso a um nodo

Acessar para  **Consulta a informações do nodo**
Alteração de algum campo de informação do nodo

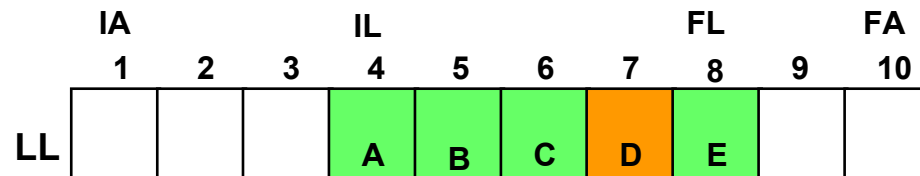
Acesso a um nodo

O nodo a ser acessado pode ser identificado por

- sua ordem na lista **Ex: 2º nodo**



- através de seu conteúdo **Ex: nodo de conteúdo “D”**



Algoritmo:

Acessar “k-ésimo” nodo da LL implementada sobre Arranjo

Algoritmo **AcessarKLLArr**

Entradas: LL (TipoLista)
IL, FL (inteiro)
K (inteiro)

Saídas: InfoNodo (TipoNodo)
Sucesso (lógico)

início

se $(K \leq 0)$ ou $(K > FL - IL + 1)$ ou $(IL = 0)$
então Sucesso $\leftarrow$ falso

senão início

InfoNodo $\leftarrow$ LL[IL+K-1]

Sucesso $\leftarrow$ verdadeiro

fim

fim

Algoritmo:

Fornece a posição na LL do nodo com um determinado Valor

Algoritmo PosValLLArr

Entradas: LL (TipoLista)
 IL, FL (inteiro)
 ValBuscado (TipoValor)

Saída: Posição (inteiro)

Variáveis auxiliares:

 I (inteiro)
 Achou (lógico)

início

 Achou $\leftarrow$ falso

 Posição $\leftarrow$ 0

 I $\leftarrow$ IL

 enquanto (I $\leq$ FL) e (não Achou)

 faça se LL[I].Valor = ValBuscado

 então início

 Posição $\leftarrow$ I

 Achou $\leftarrow$ verdadeiro

 fim

 senão I $\leftarrow$ I+1

fim

TipoNodo = registro

 Valor: inteiro

 Info: TipoInfo

 fim registro